

StarForth User Guide

Running, Scripting, and Exploring StarForth

Robert A. James

Contents

Part I

First Steps

Chapter 1

Installation

1.1 Installation

StarForth builds from source on Linux (x86_64, ARM64) and experimentally on macOS and Windows (WSL2). No binary packages are distributed at this time.

1.1.1 Prerequisites

Required

- GCC $\geq$ 7.0 or Clang $\geq$ 6.0
- GNU Make
- glibc (standard builds)

Optional

- texinfo — GNU info documentation (`sudo apt-get install texinfo`)
- dpkg-dev — Debian package generation (`sudo apt-get install dpkg-dev`)
- rpm-build — RPM package generation (`sudo dnf install rpm-build`)

1.1.2 Building from Source

The standard optimized build:

```
1 make fastest
```

This produces the maximum-performance binary with ASM optimisations, direct threading, and LTO enabled. Table ?? lists all available build targets.

Target	Description	Performance tier
<code>make fastest</code>	Maximum performance build	Tier 5 (highest)
<code>make pgo</code>	Profile-guided optimisation	Tier 6 (5–15% above fastest)
<code>make all</code>	Standard optimised build	Tier 4
<code>make debug</code>	Debug build with symbols	Tier 1
<code>make minimal</code>	Minimal/embedded build	Tier 3

Table 1.1: StarForth build targets.

1.1.3 Installation Methods

System-Wide Installation (Recommended)

```
1 make fastest
2 sudo make install
```

Installs to the following paths:

- Binary: /usr/local/bin/starforth
- Config: /usr/local/etc/starforth/init.4th
- Man page: /usr/local/share/man/man1/starforth.1
- Documentation: /usr/local/share/doc/starforth/
- Info: /usr/local/share/info/starforth.info

Custom Prefix

```
1 make fastest
2 make install PREFIX=$HOME/.local
```

Debian Package

```
1 make deb
2 sudo dpkg -i ../starforth_1.1.0-1_amd64.deb
```

RPM Package

```
1 make rpm
2 sudo rpm -ivh ~/rpmbuild/RPMS/x86_64/starforth-1.1.0-1.x86_64.rpm
```

1.1.4 Platform-Specific Notes

x86_64 Linux

Standard installation works without modification:

```
1 make fastest
2 sudo make install
```

ARM64 (Raspberry Pi 4 and compatible)

Native build on ARM64:

```
1 make rpi4
2 sudo make install
```

Cross-compile from x86_64:

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make rpi4-cross
3 scp build/starforth user@arm64-device:~/
```

macOS (Experimental)

```

1 xcode-select --install
2 make fastest CC=clang
3 sudo make install

```

Windows (WSL2)

```

1 # Inside WSL2 (Ubuntu):
2 make fastest
3 make install PREFIX=$HOME/.local
4 echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
5 source ~/.bashrc

```

1.1.5 Verifying the Installation

```

1 starforth --version
2 starforth --run-tests
3 starforth

```

Quick functional check:

```

1 starforth -c ':_HELLO_"_Hello, _World!"_CR;_HELLO_BYE'
2 starforth --benchmark 10000

```

1.1.6 Uninstallation

```

1 # From source install:
2 sudo make uninstall
3
4 # From Debian package:
5 sudo dpkg -r starforth
6
7 # From RPM package:
8 sudo rpm -e starforth

```

1.1.7 Troubleshooting

gcc: command not found

```

1 # Debian/Ubuntu:
2 sudo apt-get install build-essential
3
4 # Fedora/RHEL:
5 sudo dnf groupinstall "Development Tools"

```

starforth: command not found after installation

```

1 which starforth
2 export PATH=$HOME/.local/bin:$PATH

```

Cannot find init.4th

```

1 export STARFORTH_INIT=/usr/local/etc/starforth/init.4th

```

1.1.8 Building the Documentation

Generate the PDF manual (LaTeX $\rightarrow$ PDF):

```
1 make book
```

Generate HTML documentation (single-page and multi-page):

```
1 make book-html
```

Chapter 2

Quick Start

2.1 Quick Start

Build the fastest binary for the current platform:

```
1 make fastest
```

2.1.1 Common Build Commands

Command	What it does	When to use
<code>make fastest</code>	Maximum speed build	Production, benchmarking
<code>make fast</code>	Fast without LTO	Development, debugging
<code>make debug</code>	Debug build (-g -O0)	Debugging with GDB
<code>make benchmark</code>	Run full benchmark suite	Performance testing
<code>make help</code>	List all options	Reference

Table 2.1: Common StarForth build commands.

2.1.2 Platform-Specific Builds

x86_64 Linux

```
1 make fastest          # Auto-detects x86_64; builds with ASM
   optimisations
2 ./build/starforth
```

Raspberry Pi 4 (native)

```
1 make fastest          # Auto-detects ARM64; optimised for Cortex
   -A72
2 ./build/starforth
```

Raspberry Pi 4 (cross-compile from x86_64)

Requires gcc-aarch64-linux-gnu:

```
1 make rpi4-cross       # Builds ARM64 binary with inline ASM
2 scp build/starforth pi@raspberrypi.local:~/
```

The resulting binary is statically linked and requires no runtime dependencies on the target device.

2.1.3 Performance Modes

Build targets are ranked by output speed:

1. **make pgo** — Profile-Guided Optimisation; 5–15% faster than **fastest**; requires two build passes (3–5 minutes).
2. **make fastest** — Recommended for production; ASM optimisations, direct threading, LTO.
3. **make fast** — ASM + direct threading; no LTO; easier to instrument.
4. **make turbo** — ASM optimisations only; no direct threading.
5. **make all** — Standard build (-O2).
6. **make debug** — No optimisations (-O0); full debug symbols.

2.1.4 INIT System

At startup StarForth automatically loads `./conf/init.4th`, which defines the foundational word set. No configuration is required:

```
1 ./build/starforth
2 ok>          # Words from init.4th are immediately available
```

`init.4th` defines words, installs a dictionary fence to protect them from `FORGET`, and zeros blocks for user use.

2.1.5 Testing and Benchmarking

```
1 make bench          # Quick benchmark (1 million operations)
2 make benchmark      # Full benchmark suite
3 make test           # Run full test suite (936+ tests)
```

Typical performance for 1 million stack operations:

Build type	x86_64	ARM64 (Raspberry Pi 4)
Debug	~800 ms	~1200 ms
Standard	~250 ms	~380 ms
Fastest	~60 ms	~95 ms
PGO	~50 ms	~80 ms

Table 2.2: StarForth benchmark timings (1M stack operations).

2.1.6 Flags Enabled by **make fastest**

2.1.7 Advanced Build Options

Custom compiler flags:

```
1 make fastest CFLAGS="$(make -s print-cflags)_-DCUSTOM_FLAG"
```

Flag	Effect
-O3	Maximum compiler optimisation
-march=native (x86_64)	Use all host CPU features
-march=armv8-a+crc+simd (ARM64)	ARMv8 with NEON
-DUSE_ASM_OPT=1	Assembly optimisations
-DUSE_DIRECT_THREADING=1	Direct-threaded interpreter
-flto	Link-Time Optimisation
-funroll-loops	Loop unrolling
-finline-functions	Aggressive function inlining
-fomit-frame-pointer	Free register for execution

Table 2.3: Compiler flags active in `make fastest`.

Static binary:

```
1 make fastest LDFLAGS="-static -s"
```

Alternate compiler:

```
1 make fastest CC=clang
```

2.1.8 Troubleshooting

Illegal instruction error The host CPU does not support a required optimisation. Use a conservative baseline:

```
1 make fastest CFLAGS="$(BASE_CFLAGS) -O3 -march=x86-64 -DUSE_ASM_OPT=1"
```

Build fails outright Verify the standard build first, then re-enable optimisations incrementally:

```
1 make clean && make all
2 make clean && make fast
3 make clean && make fastest
```

Slow performance Confirm optimisation symbols are present in the binary:

```
1 file build/starforth
2 nm build/starforth | grep vm_push_asm
```


Part II

The StarForth REPL

Chapter 3

Interactive Mode

Chapter 4

Running Files and Blocks

Chapter 5

The Emergency Console

Part III

FORTH-79 Word Reference

Chapter 6

Stack Operations

Chapter 7

Arithmetic and Logic

Chapter 8

Memory and Variables

Chapter 9

Control Flow

Chapter 10

String and I/O Words

Chapter 11

Defining Words

Chapter 12

Block Words

Chapter 13

Fixed-Point Arithmetic: Q48.16

Part IV

StarForth Extensions

Chapter 14

Physics Diagnostics

Chapter 15

Heartbeat and Adaptive Tuning

15.1 Heartbeat CSV Export

The heartbeat subsystem emits real-time multivariate VM metrics to `stderr` in CSV form, exposing StarForth's adaptive runtime to external analysis. Each row carries eleven performance indicators sampled once per heartbeat tick — nominally every millisecond — producing a continuous time series of the running system's internal state.

The design favours streaming over buffering. Metrics are read directly from VM state with no intermediate copy, each row is flushed immediately so that downstream tools see data as it is produced, and the stream is confined to `stderr` to keep `stdout` reserved for test output. The emitting thread is a background `pthread` decoupled from the main interpreter, so capture does not perturb execution timing beyond a fixed, small cost per tick.

15.1.1 CSV Format

The VM emits raw rows only; column names are supplied separately by analysis tooling. The reference header is:

```
1 tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,
   bucket_hits_delta,
2 word_executions_delta,hot_word_count,avg_word_heat>window_width,
3 predicted_label_hits,estimated_jitter_ns
```

The *Status* column below reflects which fields carry live values and which currently emit a placeholder zero pending delta-tracking work.

15.1.2 Capturing Metrics

The simplest capture redirects `stderr` to a file and discards `stdout`:

```
1 # Capture metrics, discard test output
2 ./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null
3
4 # Keep both streams, separated
5 ./build/amd64/fastest/starforth --doe 1>test_output.txt 2>heartbeat.
   csv
```

Because rows are flushed as they are produced, the stream can be monitored live — for example piped through `csvlook` from `csvkit`, or followed with `tail -f`. For post-processing, prepend the reference header and load the result into R or pandas:

```
1 { echo "tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,\
2 bucket_hits_delta,word_executions_delta,hot_word_count,avg_word_heat
   ,\
```

Table 15.1: Heartbeat CSV columns

Column	Type	Unit	Description	Status
<code>tick_number</code>	<code>uint32</code>	count	Monotonic tick counter since VM start	live
<code>elapsed_ns</code>	<code>uint64</code>	ns	Time since VM initialization	live
<code>tick_interval_ns</code>	<code>uint64</code>	ns	Time since previous tick (actual vs. nominal 1 ms)	live
<code>cache_hits_delta</code>	<code>uint32</code>	count	Hotwords cache hits since last tick	TODO
<code>bucket_hits_delta</code>	<code>uint32</code>	count	Bucket-level cache hits since last tick	TODO
<code>word_executions_delta</code>	<code>uint32</code>	count	FORTH word executions since last tick	TODO
<code>hot_word_count</code>	<code>uint64</code>	count	Words at or above the heat promotion threshold (≥ 10)	live
<code>avg_word_heat</code>	<code>double</code>	heat	Mean execution heat across the dictionary ($\mathbb{Q}_{48.16} \rightarrow \text{double}$)	live
<code>window_width</code>	<code>uint32</code>	entries	Current rolling window effective size (adaptive)	live
<code>predicted_label_hits</code>	<code>uint32</code>	count	Successful pipelining speculation hits	TODO
<code>estimated_jitter_ns</code>	<code>double</code>	ns	Absolute deviation from the nominal 1 ms interval	live

```

3 window_width,predicted_label_hits,estimated_jitter_ns"; \
4 cat heartbeat.csv; } > analysis.csv

```

15.1.3 Analysis Use Cases

The eleven-dimensional series supports three broad classes of analysis. For dynamics modeling, the trajectory admits phase-space reconstruction to locate attractor basins in the adaptive parameter space, Lyapunov-exponent estimation to quantify stability, and mutual-information analysis to discover coupling between metrics. For tuning validation, window-width adaptation can be correlated against prediction accuracy, heat-decay behaviour against stale word accumulation, and jitter against thread-scheduling effects. For workload profiling, the same data yields hot-word churn rate, execution density (`word_executions_delta` over `tick_interval_ns`), and cache efficiency relative to working-set size.

15.1.4 Implementation

Capture is performed by `heartbeat_capture_tick_snapshot()`, which reads the monotonic tick counter, computes elapsed and inter-tick time, walks the dictionary to count hot words and their mean heat, samples the rolling window size, and estimates jitter as the absolute difference between the actual and nominal interval. The walk is linear in dictionary size (on the order of a few hundred words). Emission is performed by `heartbeat_emit_tick_row()`, which formats the eleven values and flushes `stderr` immediately. Both are invoked once per cycle from the background heartbeat thread.

The thread runs without the original 50 ms startup delay, so metrics begin flowing immediately even for short-running DoE workloads. The trade-off is that the first few ticks may show transient values during the word-registration phase.

15.1.5 Known Limitations

Three groups of fields are not yet wired to live deltas and currently emit zero: the cache, bucket, and word-execution counters all require per-tick baseline tracking in the heartbeat state. Separately, the source notes an uninitialized `run_start_ns`, which can produce implausibly large `elapsed_ns` values in the first ticks until the field is seeded at VM initialization, and a missing aggregation of per-word prefetch hits behind `predicted_label_hits`.

The runtime cost is small: emission adds well under one percent CPU at the nominal tick rate, the snapshot is a stack-allocated structure of under a hundred bytes, and the output volume is on the order of a hundred kilobytes per second of execution.

Chapter 16

Design of Experiments Mode

Part V

Running on Hardware

Chapter 17

Linux and POSIX Platforms

Chapter 18

Raspberry Pi 4

18.1 Building StarForth on Raspberry Pi 4

This guide covers building and tuning StarForth for the Raspberry Pi 4 with its ARM64 assembly optimizations. The board pairs a Broadcom BCM2711 (quad-core Cortex-A72 at 1.5 GHz, ARMv8-A) with 32 KB of L1 instruction and data cache per core, 1 MB of shared L2, and LPDDR4-3200 memory; NEON and CRC32 are both present.

18.1.1 Building

The Pi 4 can be built natively or cross-compiled. A native build installs `build-essential`, clones the repository, and compiles with the Cortex-A72 tuning flags:

```
1 make CFLAGS="-std=c99 -O3 -march=armv8-a+crc+simd -mtune=cortex-a72 -\
2   -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -Iinclude -\
3   -Isrc/word_source -Isrc/test_runner/include"
```

Cross-compiling from x86_64 Linux installs `gcc-aarch64-linux-gnu`, builds with `CC=aarch64-linux-gnu-gcc` and `LDFLAGS="-static"`, and copies the static binary to the board over `scp`. The repository provides convenience targets: `rpi4` for a native build, `rpi4-cross` for a static cross-build, and `rpi4-perf` for the maximum-performance build with direct threading and LTO.

18.1.2 Architecture Detection and Code Integration

A detection header (`include/arch_detect.h`) selects the right optimization headers and primitive macros at compile time — `x86_64`, `AArch64`, or a portable fallback with a warning. Stack management, arithmetic words, and dictionary lookup each keep an assembly path guarded by `USE_ASM_OPT` alongside a C path, so the architecture-agnostic `vm_push_opt/vm_pop_opt` macros resolve to the best available implementation.

18.1.3 Performance Tuning

Three techniques dominate on the Cortex-A72. Aligning the VM struct to the 64-byte cache line and grouping hot data (stacks and pointers) away from cold data reduces false sharing. Prefetching the next dictionary entry while comparing the current one hides traversal latency. And NEON block copy moves 64 bytes per iteration for bulk memory operations, falling back to scalar copy for the remainder.

18.1.4 Benchmarking

Reliable measurement requires pinning the CPU to performance mode and watching temperature. The governor is set with:

```

1 echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/
  scaling_governor
2 watch -n 1 vcgencmd measure_temp
3 vcgencmd get_throttled          # 0x0 means no throttling occurred

```

A benchmark script builds an -O2 baseline and the optimized `rpi4-perf` binary, runs a million iterations each of stack, arithmetic, and logic loops, and reports the timing difference along with final temperature and throttling status. The `perf` tool supplies cache-reference, cache-miss, branch, and branch-miss counters for deeper analysis.

18.1.5 Thermal and Power

Sustained workloads need cooling. A passive aluminum or copper heatsink covering the entire CPU with a 1 mm thermal pad is the minimum; a 5 V 30 mm fan, optionally GPIO-controlled, handles continuous load. Power draw runs 2–3 W idle, 4–5 W under light load, and 7–8 W with all cores busy, plus 2–3 W for peripherals — the official 5 V/3 A supply is recommended. Overclocking through `/boot/config.txt` is possible but requires adequate cooling and may void the warranty.

18.1.6 Storage and Deployment

For the SD card, choose UHS-I or better with an A2 application-performance rating; brands with strong random I/O include Samsung EVO Plus, SanDisk Extreme, and Kingston Canvas React. For production, a systemd service unit runs StarForth on boot with automatic restart on failure; boot time can be trimmed by disabling unused features and the splash screen in `/boot/config.txt`; and standard hardening (system updates, a UFW firewall, and key-only SSH) rounds out a deployment.

18.1.7 Troubleshooting

- *Illegal instruction at runtime* — the build exceeded the host’s feature set; rebuild with the conservative `-march=armv8-a`.
- *Cross-compiled binary will not run* — confirm it is statically linked (`file build/starforth`); rebuild with `-static` otherwise.
- *Slower than expected* — verify the governor is `performance` rather than `ondemand` and check `vcgencmd get_throttled` for thermal throttling.
- *Out of memory* — reduce `VM_MEMORY_SIZE` or add swap.

Chapter 19

LithosAnanke Bare-Metal Boot

Appendix A

Build Flags Reference

A.1 Build Options Reference

StarForth exposes a comprehensive set of build-time configuration options controlling optimizations, feature selection, platform targeting, and debugging. All options are passed as preprocessor defines via `CFLAGS` or as Makefile variables. Architecture detection runs automatically at compile time through `arch_detect.h`.

A.1.1 Performance Optimization Options

`USE_ASM_OPT`

Enables hand-optimized, architecture-specific assembly implementations for performance-critical operations. Type: integer (0 or 1). Default: 0.

Affected routines include:

- `vm_push` / `vm_pop` — data stack operations
- `vm_rpush` / `vm_rpop` — return stack operations
- `vm_add_check_overflow` / `vm_sub_check_overflow` — arithmetic with overflow detection
- `vm_mul` / `vm_div` — multiply and divide
- `vm_strcmp_asm` / `vm_memcpy_asm` — string and memory operations
- `vm_min_asm` / `vm_max_asm` — conditional-move optimizations
- `vm_abs_asm` — absolute value (ARM64)
- `vm_clz` / `vm_ctz` — count leading/trailing zeros (ARM64)
- `vm_prefetch` / `vm_prefetch_stream` — cache prefetching (ARM64)

Typical performance impact: $\approx 20\text{--}25\%$ on `x86_64`; $\approx 15\text{--}20\%$ on ARM64. Enabled automatically by the `fastest`, `fast`, `turbo`, and `pgo` targets.

`USE_DIRECT_THREADING`

Switches the inner interpreter from indirect threading (the FORTH-79 default) to direct threading using GCC computed-goto (`&&label` / `goto *ptr`). In direct-threaded mode each word's code field points directly to native code, eliminating one indirection level per dispatch. Type: integer (0 or 1). Default: 0.

Requirements: GCC or Clang; must be combined with `USE_ASM_OPT=1`. Full support is provided for x86_64 (`vm_inner_interp_asm.h`) and ARM64 (`vm_inner_interp_arm64.h`).

Typical performance impact: $\approx 30\text{--}40\%$ over indirect threading when combined with `USE_ASM_OPT=1`. Enabled by `fastest`, `fast`, and `pgo` targets.

NDEBUG

Standard C99 macro. When defined, all `assert()` calls compile to nothing, and StarForth additionally suppresses debug logging in hot paths. Default: undefined (assertions active). Typical impact: $\approx 5\text{--}10\%$ speedup.

STARFORTH_PERFORMANCE

Experimental flag that trades safety for throughput. Reduces stack-bounds checking in hot paths (DUP, SWAP, etc.) and applies `UNLIKELY()` branch hints, assuming well-formed Forth input.

Caution: stack overflows may go undetected. Use only on code that has been exhaustively tested. Typical additional gain: $\approx 5\text{--}8\%$ over standard `-O3`. Relevant sources: `src/word_source/stack_words.c:5` (DUP), `src/word_source/stack_words.c:117` (SWAP).

A.1.2 Architecture Options

Architecture detection is automatic via `arch_detect.h`; manual overrides are available for cross-compilation scenarios.

ARCH_X86_64

Targets x86-64 (AMD64 / Intel 64). Auto-detected from compiler predefined macros (`__x86_64__`, `_M_X64`, `__amd64__`). Enables x86_64 assembly optimizations, SSE4.2 string instructions, and the x86_64 direct-threading header.

ARCH_ARM64

Targets AArch64 (ARMv8-A). Auto-detected from `__aarch64__`, `_M_ARM64`, and `__arm64__`. Enables NEON SIMD instructions, ARM64-specific instructions (CLZ, CTZ, RBIT, REV), ARM64 assembly optimizations, and ARM64 direct threading.

Native ARM64 build: `make rpi4`. Cross-compile from x86_64: `make rpi4-cross`.

ARCH_NAME

Human-readable string set automatically: `"x86_64"`, `"ARM64"`, or `"Unknown"`. Displayed by `make help`.

On an unrecognized architecture the build emits a compiler warning and falls back to pure-C implementations; no assembly optimizations are loaded.

A.1.3 Platform / Target Options

STARFORTH_MINIMAL

Enables minimal/freestanding build suitable for bare-metal or microkernel environments. When defined: ANSI color codes are suppressed in logging; I/O is reduced to minimal output; the build links with `-nostdlib -ffreestanding`. Enabled by `make minimal` and `make 14re`.

L4RE_TARGET

Targets the L4Re microkernel operating system. Enables the L4Re block-storage backend, ROMFS file access in place of POSIX, and implies `STARFORTH_MINIMAL=1`. Status: partially implemented (ROMFS stub present). See `src/word_source/starforth_words.c:225` and `include/blkio_fa`

A.1.4 Debugging and Profiling Options**DEBUG**

Debug builds compile with `-O0 -g -DDEBUG`: no optimization, full debugging symbols, all assertions enabled, verbose logging throughout. Enabled by `make debug`.

PROFILE_ENABLED

Activates StarForth's built-in profiler. Type: integer (0 or 1). Default: 0. The profiler tracks per-word execution counts, call-graph relationships (caller/callee), stack operation counts, and total instruction counts. Overhead: $\approx 10\text{--}15\%$.

```
1 ./build/starforth --profile 2          # enable at depth 2
2 ./build/starforth --profile-report # print results
```

See `include/profiler.h`.

STRICT_PTR

Controls the Forth address-to-C pointer mapping. Type: integer (0 or 1). Default: 1.

- `STRICT_PTR=1` — Forth addresses are real pointers (`cell_t = uintptr_t`).
- `STRICT_PTR=0` — Forth addresses are indices/offsets (segmented memory models).

Disable only for benchmarking or non-hosted targets. See `src/vm_api.c:202` and `Makefile:37`.

A.1.5 Block System Options

- `BLKCFG_DEFAULT_FBS` — default Forth block size in bytes (default: 1024). See `include/blkcfg.h`.
- `BLKCFG_PATH_MAX` — maximum path length for block-storage files (default: 4096).
- `BLK_FORTH_SYS_RESERVED` — number of system-reserved RAM blocks, hidden from user access (default: 32, blocks 0–31).
- `BLK_DISK_SYS_RESERVED` — number of system-reserved disk blocks (default: 32, PBN 1024–1055).

A.1.6 Memory Configuration Options

- `DATA_STACK_SIZE` — data stack depth in cells (default: 256). See `include/vm.h`.
- `RETURN_STACK_SIZE` — return stack depth in cells (default: 256).
- `STARFORTH_STATE_BYTES` — total state-buffer size for minimal builds (default: 8192). See `src/main.c:40–42`.
- `SF_FC_BUCKETS` — number of forget-chain hash buckets; defined in `vm.h`.

A.1.7 Feature Flags

STARFORTH_ANSI

Enables ANSI escape sequences for terminal control in the block editor (LIST, EDIT): screen clear (`\x1b[2J`), cursor positioning, and colored line numbers. See `src/word_source/editor_words.c:74,188,20`

VM_HAS_CURRENT_ENTRY

Enables the `vm->current_entry` field, which tracks the currently-compiling word. Required for the SEE decompiler, recursive word definitions, and dictionary introspection. Default: defined (enabled).

A.1.8 Build System Variables

- **CC** — C compiler. Default: `gcc`. Also accepts `clang` and `aarch64-linux-gnu-gcc`.
- **CFLAGS** — compiler flags. Extends Makefile `BASE_CFLAGS`.
- **LDFLAGS** — linker flags. Default includes `-Wl,-gc-sections -s -flto=auto -fuse-linker-plugin -static`.
- **MINIMAL** — integer (0 or 1). Setting `MINIMAL=1` enables `STARFORTH_MINIMAL` and adds `-nostdlib -ffreestanding`.
- **ASM** — integer (0 or 1). When set to 1, the build emits `.s` assembly listings alongside `.o` object files.

A.1.9 Build Configuration Matrix

Table A.1: Common build configurations

Target	USE_ASM_OPT	USE_DIRECT_THREADING	NDEBUG	Opt level	Use case
<code>debug</code>	0	0	No	<code>-O0 -g</code>	Development
<code>all</code>	1	0	No	<code>-O2</code>	Default build
<code>turbo</code>	1	0	Yes	<code>-O3 -flto</code>	Fast, no debug
<code>fast</code>	1	1	Yes	<code>-O3</code>	Fast, readable
<code>fastest</code>	1	1	Yes	<code>-O3 -flto</code>	Maximum speed
<code>pgo</code>	1	1	Yes	<code>-O3 -fprofile-use</code>	Absolute fastest
<code>minimal</code>	0	0	No	<code>-O2</code>	Embedded
<code>l4re</code>	0	0	No	<code>-O2</code>	L4Re microkernel
<code>profile</code>	0	0	No	<code>-O1 -g</code>	Profiling

A.1.10 Optimization Level Guidelines

- **-O0 -g (development)** — fastest compile, slowest runtime ($\approx 10\times$ below optimized); full symbols, all assertions. Use for initial development and crash debugging.
- **-O2 (balanced)** — moderate compile overhead; 3–4 $\times$ runtime gain over `-O0`. Default production-test builds.
- **-O3 (high performance)** — aggressive inlining and vectorization; 5–6 $\times$ over `-O0`. Production releases.

- **-O3 -flto -march=native -fprofile-use (maximum)** — multi-stage build; 7–8× over -O0. Release and benchmark artifacts. Use `make pgo`.

A.1.11 Usage Examples

Maximum performance native build

```
1 make clean && make pgo
```

Produces a PGO-optimized binary with assembly optimizations, direct threading, and link-time optimization.

Debug build with profiler

```
1 make clean && make profile
2 ./build/starforth --profile 3 --profile-report
```

Cross-compile for Raspberry Pi 4

```
1 make rpi4-cross
2 scp build/starforth pi@raspberrypi.local:~/
```

Produces a statically linked ARM64 binary optimized for Cortex-A72.

Minimal embedded build

```
1 make minimal
```

Produces a freestanding binary with no libc dependencies.

AMD Zen 3 custom build

```
1 make clean
2 make CFLAGS="$(BASE_CFLAGS) -O3 -march=znver3 \
3     -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DNDEBUG" \
4     LDFLAGS="-flto -s"
```

Generate assembly listings

```
1 make asm
2 less build/stack_management.s
```

A.1.12 Compiler Support

- **GCC** — recommended. Minimum GCC 7.0; GCC 11.0+ preferred for improved PGO and ARM64 code generation.
- **Clang** — minimum Clang 10.0; computed-goto and all assembly optimizations supported.
- **ARM64 cross-compilation from x86_64:**

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make CC=aarch64-linux-gnu-gcc rpi4-cross
```

Static linking (`-static`) is recommended for cross-compiled binaries.

A.1.13 Performance Tuning Checklist

For maximum throughput:

- Use `make pgo` (profile-guided optimization).
- Enable `USE_ASM_OPT=1` and `USE_DIRECT_THREADING=1`.
- Define `NDEBUG`.
- Pass `-O3 -flto -march=native`.
- Add `-fno-plt -fno-semantic-interposition` to reduce indirection.
- Add `-funroll-loops -finline-functions` for code expansion.
- Link statically (`-static`) to eliminate PLT overhead.
- Strip symbols (`-s`) to reduce binary size.

Expected result: $\approx 7\text{--}8\times$ faster than a debug build; $\approx 1.2\text{--}1.5\times$ faster than a plain `-O3` build.

A.1.14 Troubleshooting

“Unknown architecture” warning The build falls back automatically to pure-C implementations. Assembly optimizations are disabled. To add support for a new architecture, extend `arch_detect.h`.

Direct threading shows no performance gain Verify that both `USE_DIRECT_THREADING=1` and `USE_ASM_OPT=1` are set, that the target is `x86_64` or `ARM64`, and that the compiler is GCC or Clang. At runtime, `WORDS-INFO` reports whether direct threading is active.

PGO coverage mismatch warnings Run `make clean && make pgo` to force a full clean two-stage rebuild. Stale coverage data from a prior instrumentation pass causes these warnings.

Appendix B

Glossary

